PRACTICAL ASSIGNMENT

One-to-one relationships in MongoDB.

Let's go step by step with a simple example.

Example: User Profile System

We'll have:

- **Users** collection
- **Profiles** collection
  (each user has exactly one profile)

Method 1: Embedding (One-to-One inside the same document)

Good when the related data is small and always needed with the parent.

```
db.users.insertOne({
  _id: ObjectId("650f1c9f1a23b8e6f1234567"),
  username: "johndoe",
  email: "johndoe@example.com",
  profile: {
    age: 28,
    gender: "Male",
    bio: "Software developer from NYC"
  }
});
```

Now fetching a user also gets their profile:

```
db.users.findOne({ username: "johndoe" });
```

Method 2: Referencing (Separate collections)

Good when the related data is big or not always required.

Users collection

```
db.users.insertOne({
  _id: ObjectId("650f1c9f1a23b8e6f1234567"),
  username: "janedoe",
  email: "janedoe@example.com",
  profileId: ObjectId("650f1ca41a23b8e6f7654321") // reference to
profile
});
```

Profiles collection

```
db.profiles.insertOne({
  _id: ObjectId("650f1ca41a23b8e6f7654321"),
  age: 25,
  gender: "Female",
  bio: "Graphic designer who loves painting"
});
```

Query with `$lookup` (join):

```
db.users.aggregate([
  {
    $lookup: {
      from: "profiles",
      localField: "profileId",
      foreignField: "_id",
      as: "profile"
    }
  },
  { $unwind: "$profile" }
]);
```

✅ **Practice Tip:**
Try both approaches and see how queries behave.

- Embedding = faster for always-needed small data.

- Referencing = more flexible, avoids big document size.

🎯 Task: Students & Passports

Each **student** has exactly **one passport**.
We'll create two collections: `students` and `passports`.

Step 1: Insert Students

```
db.students.insertMany([
  {
    _id: ObjectId("651000000000000000000001"),
    name: "Alice Johnson",
    age: 22,
    passportId: ObjectId("651100000000000000000001") // reference
  },
  {
    _id: ObjectId("651000000000000000000002"),
    name: "Bob Smith",
    age: 24,
    passportId: ObjectId("651100000000000000000002") // reference
  }
]);
```

Step 2: Insert Passports

```
db.passports.insertMany([
  {
    _id: ObjectId("651100000000000000000001"),
    number: "A1234567",
    country: "USA",
    expiry: new Date("2030-05-01")
  },
  {
```

```
    _id: ObjectId("651100000000000000000002"),
    number: "B7654321",
    country: "Canada",
    expiry: new Date("2029-11-20")
  }
]);
```

Step 3: Query with `$lookup` (Join)

```
db.students.aggregate([
  {
    $lookup: {
      from: "passports",
      localField: "passportId",
      foreignField: "_id",
      as: "passport"
    }
  },
  { $unwind: "$passport" }
]);
```

✅ This will return each student with their passport details in one document.

Step 4 Queries: Try yourself

1. Insert a **third student** and a passport.
2. Query only students whose passports expire **after 2029**.
3. (Bonus) Embed passport info inside the student document instead of referencing.

🔹 METHOD Referenced (One-to-One)

students

```
{
  _id: ObjectId("651000000000000000000001"),
  name: "Alice Johnson",
  age: 22,
  passportId: ObjectId("651100000000000000000001")
}
```

passports

```
{
  _id: ObjectId("651100000000000000000001"),
  number: "A1234567",
  country: "USA",
  expiry: ISODate("2030-05-01")
}
```

🔹 Practice Queries

1. Get all students with their passport (JOIN)

```
db.students.aggregate([
  {
    $lookup: {
      from: "passports",
      localField: "passportId",
      foreignField: "_id",
      as: "passport"
    }
  },
  { $unwind: "$passport" }
]);
```

---

2. Get one student with their passport (by name)

```
db.students.aggregate([
  { $match: { name: "Alice Johnson" } },
  {
    $lookup: {
      from: "passports",
      localField: "passportId",
      foreignField: "_id",
      as: "passport"
    }
  },
  { $unwind: "$passport" }
]);
```

---

3. Find all students whose passport is from USA

```
db.students.aggregate([
  {
    $lookup: {
      from: "passports",
      localField: "passportId",
      foreignField: "_id",
      as: "passport"
    }
  },
  { $unwind: "$passport" },
  { $match: { "passport.country": "USA" } }
]);
```

---

4. Find students whose passports expire after 2029

```
db.students.aggregate([
  {
    $lookup: {
      from: "passports",
      localField: "passportId",
```

```
      foreignField: "_id",
      as: "passport"
    }
  },
  { $unwind: "$passport" },
  { $match: { "passport.expiry": { $gt: ISODate("2029-01-01") } } }
]);
```

---

5. Project only student name + passport number

```
db.students.aggregate([
  {
    $lookup: {
      from: "passports",
      localField: "passportId",
      foreignField: "_id",
      as: "passport"
    }
  },
  { $unwind: "$passport" },
  { $project: { _id: 0, name: 1, "passport.number": 1 } }
]);
```

---

6. Update passport expiry date for one student

```
db.passports.updateOne(
  { number: "A1234567" },
  { $set: { expiry: ISODate("2035-12-31") } }
);
```

---

7. Delete a student (and optionally their passport)

```
// Delete student only
db.students.deleteOne({ name: "Bob Smith" });

// Delete passport too
```

```
db.passports.deleteOne({ number: "B7654321" });
```

🔹 (METHOD Embedded (One-to-One)

Here we keep everything in the `students` collection only:

```
db.students.insertMany([
  {
    _id: ObjectId("651000000000000000000001"),
    name: "Alice Johnson",
    age: 22,
    passport: {
      number: "A1234567",
      country: "USA",
      expiry: ISODate("2030-05-01")
    }
  },
  {
    _id: ObjectId("651000000000000000000002"),
    name: "Bob Smith",
    age: 24,
    passport: {
      number: "B7654321",
      country: "Canada",
      expiry: ISODate("2029-11-20")
    }
  }
]);
```

🔹 Practice Queries (Embedded)

1. Get all students with passport info

(since passport is embedded, it's just a normal `find`)

```
db.students.find({});
```

---

2. Get one student with passport (by name)

```
db.students.findOne({ name: "Alice Johnson" });
```

---

3. Find all students whose passport is from USA

```
db.students.find({ "passport.country": "USA" });
```

---

4. Find students whose passports expire after 2029

```
db.students.find({ "passport.expiry": { $gt: ISODate("2029-01-01") }
});
```

---

5. Project only student name + passport number

```
db.students.find(
  {},
  { _id: 0, name: 1, "passport.number": 1 }
);
```

---

6. Update passport expiry date for one student

```
db.students.updateOne(
  { "passport.number": "A1234567" },
  { $set: { "passport.expiry": ISODate("2035-12-31") } }
);
```

---

7. Delete a student (passport will also be deleted automatically since it's embedded)

```
db.students.deleteOne({ name: "Bob Smith" });
```

✅ **Comparison**

- **Referenced model** → needs `$lookup` (like joins).
- **Embedded model** → simpler queries, but less flexible if passport data grows large or changes independently.

📘 One-to-One Relationship Cheatsheet(COMPARISON) (MongoDB)

EMBEDDED VERSUS REFERENCED

Data Models

🔹 Embedded (all in one collection)

```
db.students.insertOne({
  name: "Alice Johnson",
  age: 22,
  passport: {
    number: "A1234567",
    country: "USA",
    expiry: ISODate("2030-05-01")
  }
});
```

🔹 Referenced (separate collections)

```
// students
db.students.insertOne({
  name: "Alice Johnson",
  age: 22,
```

```
  passportId: ObjectId("651100000000000000000001")
});

// passports
db.passports.insertOne({
  _id: ObjectId("651100000000000000000001"),
  number: "A1234567",
  country: "USA",
  expiry: ISODate("2030-05-01")
});
```

Queries Side-by-Side

Operation	Embedded	Referenced
Get all students with passport	`db.students.find({})`	`db.students.aggregate([{ $lookup: { from: "passports", localField: "passportId", foreignField: "_id", as: "passport" } }, { $unwind: "$passport" }])`
Get one student with passport	`db.students.findOne({ name: "Alice Johnson" })`	Same `$lookup` + `$match` on name
Find students with USA passport	`db.students.find({ "passport.country": "USA" })`	`$lookup` + `$match: { "passport.country": "USA" }`
Find passports expiring after 2029	`db.students.find({ "passport.expiry": { $gt: ISODate("2029-01-01") } })`	`$lookup` + `$match: { "passport.expiry": { $gt: ISODate("2029-01-01") } }`
Project name + passport number	`db.students.find({}, { name: 1, "passport.number": 1, _id: 0 })`	`$lookup` + `$project: { name: 1, "passport.number": 1 }`

Update passport expiry	`db.students.updateOne({ "passport.number": "A1234567" }, { $set: { "passport.expiry": ISODate("2035-12-31") } })`	`db.passports.updateOne({ number: "A1234567" }, { $set: { expiry: ISODate("2035-12-31") } })`
Delete student (+passport)	`db.students.deleteOne({ name: "Bob Smith" })` (passport auto deleted)	Delete from `students`, then separately from `passports`

🔑 Key Takeaways

- **Embedded**:
  ✅ Faster + simpler queries
  ❌ Not good if passport needs independent updates often

- **Referenced**:
  ✅ More flexible, avoids huge student documents
  ❌ Requires `$lookup` joins, which are slower

One-to-Many relationships in MongoDB.

We'll take a simple example:

🎯 Example: Authors & Books

- **One Author → Many Books**

🔹 Option 1: Embedded One-to-Many (books inside author)

```
db.authors.insertMany([
  {
    _id: ObjectId("651200000000000000000001"),
    name: "J.K. Rowling",
    books: [
      { title: "Harry Potter and the Sorcerer's Stone", year: 1997 },
      { title: "Harry Potter and the Chamber of Secrets", year: 1998 }
    ]
  },
  {
    _id: ObjectId("651200000000000000000002"),
    name: "George R.R. Martin",
    books: [
      { title: "A Game of Thrones", year: 1996 },
      { title: "A Clash of Kings", year: 1998 }
    ]
  }
]);
```

🔹 Practice Queries (Embedded)

1. **Get all authors with their books**

```
db.authors.find({});
```

2. **Find books by J.K. Rowling**

```
db.authors.findOne({ name: "J.K. Rowling" }, { books: 1, _id: 0 });
```

3. **Find authors who wrote a book in 1998**

```
db.authors.find({ "books.year": 1998 });
```

4. **Project only author + book titles**

```
db.authors.find({}, { name: 1, "books.title": 1, _id: 0 });
```

5. **Add a new book to an author**

```
db.authors.updateOne(
  { name: "J.K. Rowling" },
  { $push: { books: { title: "Harry Potter and the Prisoner of
Azkaban", year: 1999 } } }
);
```

🔹 Option 2: Referenced One-to-Many (books in separate collection)

```
// Authors
db.authors.insertOne({
  _id: ObjectId("651200000000000000000010"),
  name: "J.R.R. Tolkien"
});

// Books
db.books.insertMany([
  { title: "The Hobbit", year: 1937, authorId:
ObjectId("651200000000000000000010") },
  { title: "The Fellowship of the Ring", year: 1954, authorId:
ObjectId("651200000000000000000010") }
]);
```

🔹 Practice Queries (Referenced)

1. **Get all authors with their books**

```
db.authors.aggregate([
  {
    $lookup: {
      from: "books",
      localField: "_id",
      foreignField: "authorId",
      as: "books"
    }
  }
]);
```

2. **Find all books of J.R.R. Tolkien**

```
db.authors.aggregate([
  { $match: { name: "J.R.R. Tolkien" } },
  {
    $lookup: {
      from: "books",
      localField: "_id",
      foreignField: "authorId",
      as: "books"
    }
  }
]);
```

3. **Find all authors who wrote books after 1950**

```
db.authors.aggregate([
  {
    $lookup: {
      from: "books",
      localField: "_id",
      foreignField: "authorId",
      as: "books"
    }
  },
```

```
  { $match: { "books.year": { $gt: 1950 } } }
]);
```

4. **Project only author name + book titles**

```
db.authors.aggregate([
  {
    $lookup: {
      from: "books",
      localField: "_id",
      foreignField: "authorId",
      as: "books"
    }
  },
  { $project: { _id: 0, name: 1, "books.title": 1 } }
]);
```

5. **Insert new book for existing author**

```
db.books.insertOne({
  title: "The Two Towers",
  year: 1954,
  authorId: ObjectId("651200000000000000000010")
});
```

📘 One-to-Many Relationship Cheatsheet (COMPARISON) (MongoDB)

EMBEDDED VERSUS REFERENCED

Data Models

- **Embedded (books inside author)**

```
db.authors.insertOne({
  name: "J.K. Rowling",
  books: [
    { title: "Harry Potter and the Sorcerer's Stone", year: 1997 },
    { title: "Harry Potter and the Chamber of Secrets", year: 1998 }
  ]
});
```

- **Referenced (books in separate collection)**

```
// Authors
db.authors.insertOne({
  _id: ObjectId("651200000000000000000010"),
  name: "J.R.R. Tolkien"
});

// Books
db.books.insertMany([
  { title: "The Hobbit", year: 1937, authorId:
ObjectId("651200000000000000000010") },
  { title: "The Fellowship of the Ring", year: 1954, authorId:
ObjectId("651200000000000000000010") }
]);
```

Queries Side-by-Side

Operation	Embedded	Referenced
Get all authors with books	`db.authors.find({})`	`$lookup` from `books` into `authors`
Get books of a specific author	`db.authors.findOne({ name: "J.K. Rowling" }, { books: 1 })`	`$match` on author + `$lookup` from books

Find authors who wrote a book in 1998	`db.authors.find({ "books.year": 1998 })`	`$lookup` + `$match: { "books.year": 1998 }`
Project only author + book titles	`db.authors.find({}, { name: 1, "books.title": 1 })`	`$lookup` + `$project: { name: 1, "books.title": 1 }`
Add a new book	`$push` into `books` array inside author	Insert a new doc into `books` with `authorId`
Update a book title	`db.authors.updateOne({ name: "J.K. Rowling", "books.title": "HP1" }, { $set: { "books.$.title": "Harry Potter 1" } })`	`db.books.updateOne({ title: "HP1" }, { $set: { title: "Harry Potter 1" } })`
Delete a book	`$pull` from `books` array	`db.books.deleteOne({ title: "HP1" })`

🔑 Key Takeaways

- **Embedded**
  ✅ Best when books are small in number and always fetched with author
  ❌ Hard to query/update individual books across authors

- **Referenced**
  ✅ Best when books are large, queried independently, or grow without limit
  ❌ Requires `$lookup` joins, more complex queries

PRACTICE ON AGGREGATION QUERIES WITH ONE TO ONE AND ONE TO MANY RELATIONSHIPS

Practice Exercises

We'll keep using the same collections:

`users`

```
{ "_id": 1, "name": "Alice", "country": "USA" }
{ "_id": 2, "name": "Bob", "country": "Canada" }
{ "_id": 3, "name": "Charlie", "country": "USA" }
{ "_id": 4, "name": "David", "country": "India" }
```

`profiles`

```
{ "_id": 1, "userId": 1, "age": 25 }
{ "_id": 2, "userId": 2, "age": 30 }
{ "_id": 3, "userId": 3, "age": 28 }
{ "_id": 4, "userId": 4, "age": 35 }
```

📝 Exercises

1. **Count how many users are in each country.**
   Hint: Use `$group` with `country` as `_id`.

---

2. **List all users grouped by their country.**
   Hint: `$group` + `$push` to collect names.

---

3. **Find the average age of users in each country.**
   Hint: Use `$lookup` to join `profiles`, then `$avg`.

---

4. **Get the maximum age of users per country.**
   Hint: `$max` instead of `$avg`.

---

5. **Find the number of users above age 30 in each country.**
   Hint: `$lookup`, then `$match`, then `$group`.

---

6. **Show each country with both the total number of users and the average age.**
   Hint: `$sum` + `$avg` in the same `$group`.

---

7. **For each country, return a list of `{ name, age }` objects for its users.**
   Hint: `$push: { name: "$name", age: "$profile.age" }`.

Example Dataset

`users` collection:

```
{ "_id": 1, "name": "Alice", "country": "USA" }
{ "_id": 2, "name": "Bob", "country": "Canada" }
{ "_id": 3, "name": "Charlie", "country": "USA" }
{ "_id": 4, "name": "David", "country": "India" }
```

`profiles` collection (one-to-one with `users`):

```
{ "_id": 1, "userId": 1, "age": 25 }
{ "_id": 2, "userId": 2, "age": 30 }
{ "_id": 3, "userId": 3, "age": 28 }
{ "_id": 4, "userId": 4, "age": 35 }
```

Practice Queries

1. Group users by country

```
db.users.aggregate([
  { $group: { _id: "$country", totalUsers: { $sum: 1 } } }
])
```

👉 Output: count of users per country.

---

2. Average age of users by country (joining with `profiles`)

```
db.users.aggregate([
  {
    $lookup: {
      from: "profiles",
      localField: "_id",
      foreignField: "userId",
      as: "profile"
    }
  },
  { $unwind: "$profile" },
  {
    $group: {
      _id: "$country",
      avgAge: { $avg: "$profile.age" },
      totalUsers: { $sum: 1 }
    }
  }
])
```

👉 Groups users by country, shows average age.

---

3. Find the max age in each country

```
db.users.aggregate([
  {
    $lookup: {
```

```
      from: "profiles",
      localField: "_id",
      foreignField: "userId",
      as: "profile"
    }
  },
  { $unwind: "$profile" },
  {
    $group: {
      _id: "$country",
      maxAge: { $max: "$profile.age" }
    }
  }
])
```

---

4. Count users older than 30 by country

```
db.users.aggregate([
  {
    $lookup: {
      from: "profiles",
      localField: "_id",
      foreignField: "userId",
      as: "profile"
    }
  },
  { $unwind: "$profile" },
  { $match: { "profile.age": { $gt: 30 } } },
  {
    $group: {
      _id: "$country",
      usersAbove30: { $sum: 1 }
    }
  }
])
```

---

5. Group by country and list user names

```
db.users.aggregate([
  {
    $group: {
      _id: "$country",
      users: { $push: "$name" }
    }
  }
])
```

👉 Similar to `GROUP_CONCAT()` in SQL.

⚡ These examples cover:

- Simple grouping (`$group`)
- One-to-one joins (`$lookup + $unwind`)
- Aggregate functions (`$sum`, `$avg`, `$max`)
- Filtering with `$match`

📘

Example Dataset

`customers`

```
{ "_id": 1, "name": "Alice", "country": "USA" }
{ "_id": 2, "name": "Bob", "country": "Canada" }
{ "_id": 3, "name": "Charlie", "country": "USA" }
```

`orders` (one-to-many with `customers`)

```
{ "_id": 101, "customerId": 1, "amount": 200, "status": "Shipped" }
{ "_id": 102, "customerId": 1, "amount": 150, "status": "Pending" }
{ "_id": 103, "customerId": 2, "amount": 300, "status": "Shipped" }
{ "_id": 104, "customerId": 3, "amount": 400, "status": "Pending" }
{ "_id": 105, "customerId": 3, "amount": 250, "status": "Shipped" }
```

📝 Practice Exercises

1. Count total orders per customer

👉 Equivalent to `GROUP BY customerId`.

- Expected output: `{ customerId: 1, totalOrders: 2 }`, etc.

---

2. Find total order amount per customer

👉 Use `$sum` on `amount`.

---

3. Find average order amount per customer

👉 Use `$avg` on `amount`.

---

4. Group orders by status and count how many orders are in each status

👉 Example: `{ status: "Shipped", count: 3 }`.

---

5. Find total order amount per country

👉 Join `customers` → `orders` using `$lookup`, then group by country.

---

6. Find max order amount for each customer

👉 Use `$max` on `amount`.

---

7. For each customer, return a list of all their order IDs

👉 Use `$push` inside `$group`.

---

8. For each country, count how many customers placed at least one order

👉 `$lookup` + `$group`.

---

9. For each customer, separate orders into shipped vs pending

👉 Use `$group` with `$push` and a conditional `$cond`.

---

10. For each customer, find both:

- totalOrders
- totalAmount
- avgAmount
  👉 Multiple accumulators in the same `$group`.

ANSWERS

`customers`

```
{ "_id": 1, "name": "Alice", "country": "USA" }
{ "_id": 2, "name": "Bob", "country": "Canada" }
{ "_id": 3, "name": "Charlie", "country": "USA" }
```

orders

```
{ "_id": 101, "customerId": 1, "amount": 200, "status": "Shipped" }
{ "_id": 102, "customerId": 1, "amount": 150, "status": "Pending" }
{ "_id": 103, "customerId": 2, "amount": 300, "status": "Shipped" }
{ "_id": 104, "customerId": 3, "amount": 400, "status": "Pending" }
{ "_id": 105, "customerId": 3, "amount": 250, "status": "Shipped" }
```

✅ Answers

1. Count total orders per customer

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customerId",
      totalOrders: { $sum: 1 }
    }
  }
])
```

---

2. Find total order amount per customer

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customerId",
      totalAmount: { $sum: "$amount" }
    }
  }
])
```

---

3. Find average order amount per customer

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customerId",
      avgAmount: { $avg: "$amount" }
    }
  }
])
```

---

4. Group orders by status and count how many orders are in each status

```
db.orders.aggregate([
  {
    $group: {
      _id: "$status",
      count: { $sum: 1 }
    }
  }
])
```

---

5. Find total order amount per country

```
db.customers.aggregate([
  {
    $lookup: {
      from: "orders",
      localField: "_id",
      foreignField: "customerId",
      as: "orders"
    }
  },
  { $unwind: "$orders" },
  {
    $group: {
```

```
      _id: "$country",
      totalAmount: { $sum: "$orders.amount" }
    }
  }
])
```

---

6. Find max order amount for each customer

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customerId",
      maxOrder: { $max: "$amount" }
    }
  }
])
```

---

7. For each customer, return a list of all their order IDs

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customerId",
      orders: { $push: "$_id" }
    }
  }
])
```

---

8. For each country, count how many customers placed at least one order

```
db.customers.aggregate([
  {
    $lookup: {
      from: "orders",
      localField: "_id",
      foreignField: "customerId",
```

```
      as: "orders"
    }
  },
  { $match: { "orders.0": { $exists: true } } }, // only customers
with at least 1 order
  {
    $group: {
      _id: "$country",
      totalCustomers: { $sum: 1 }
    }
  }
])
```

---

9. For each customer, separate orders into shipped vs pending

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customerId",
      shippedOrders: {
        $push: { $cond: [{ $eq: ["$status", "Shipped"] }, "$_id",
"$$REMOVE"] }
      },
      pendingOrders: {
        $push: { $cond: [{ $eq: ["$status", "Pending"] }, "$_id",
"$$REMOVE"] }
      }
    }
  }
])
```

---

10. For each customer, find both totalOrders, totalAmount, avgAmount

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customerId",
```

```
      totalOrders: { $sum: 1 },
      totalAmount: { $sum: "$amount" },
      avgAmount: { $avg: "$amount" }
    }
  }
])
```